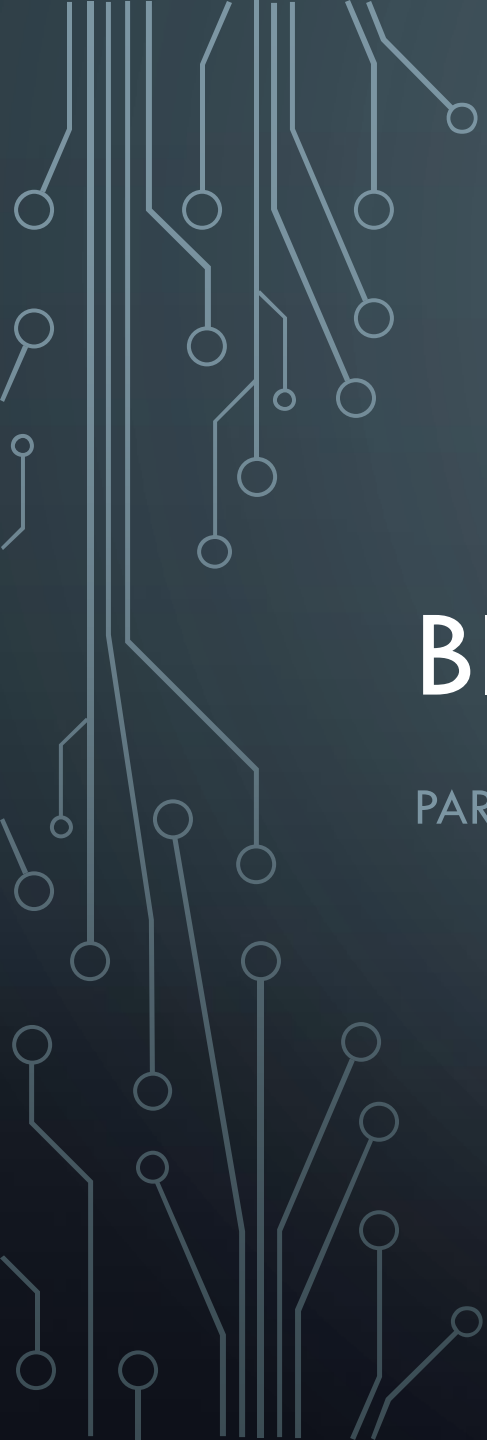


An abstract graphic on the left side of the slide, consisting of a network of white lines and circles on a dark blue background. The lines are vertical and horizontal, with circles at various points, resembling a circuit board or a data network.

BIG DATA

PARALLELISATION AND DISTRIBUTION

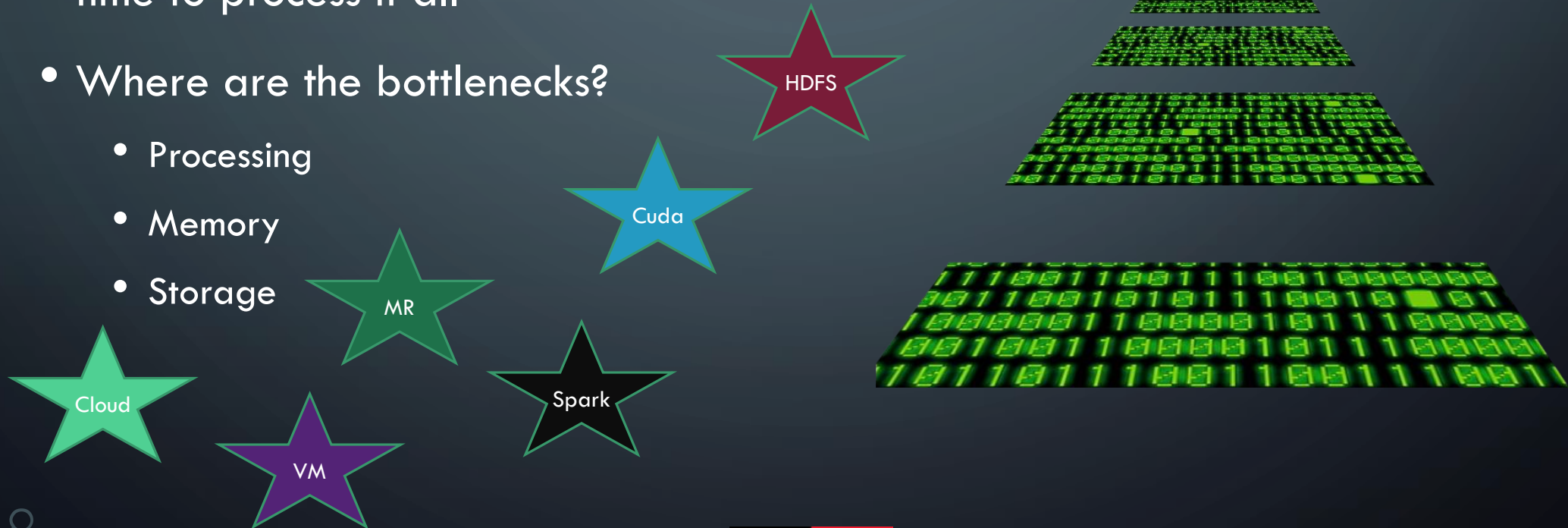
LEARNING OBJECTIVES

- At the end of this presentation, you should be able to
 - Explain what the potential bottlenecks are when processing big data;
 - Explain how these bottlenecks are typically avoided;
 - Understand how the core parts of Hadoop address the problem;
 - Assess whether MapReduce can be applied to a problem;
 - Explain the process of MapReduce.

BIG DATA AND LATENCY

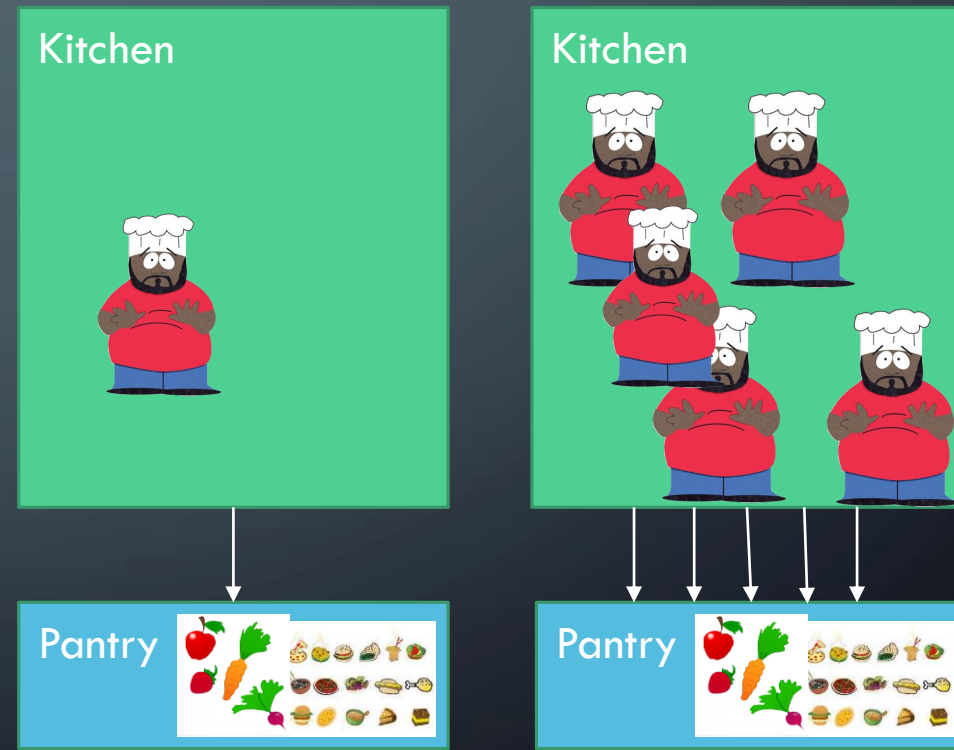
- Having a lot of data means it takes time to process it all
- Where are the bottlenecks?

- Processing
- Memory
- Storage



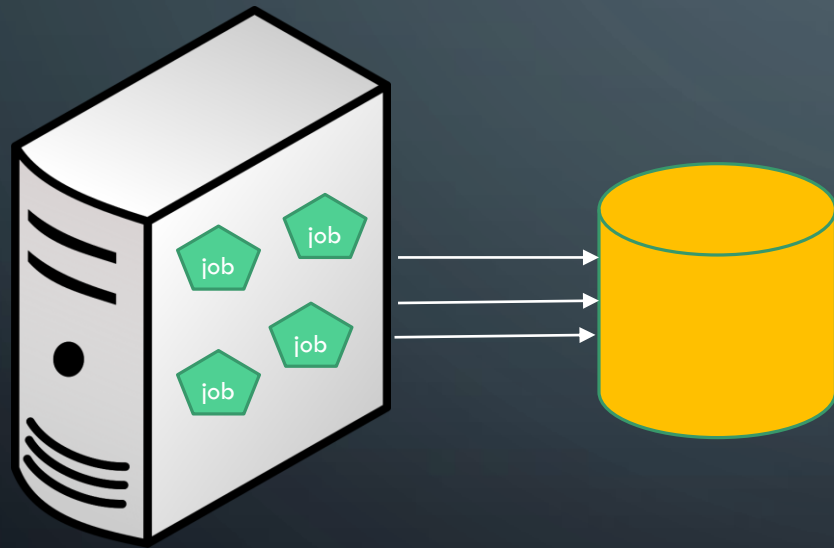
PARALLELISATION AND DISTRIBUTION

- To speed things up, first we parallelise.
 - If your restaurant is busy, a single chef won't be able to keep up.
 - If you need to hire very many chefs, the kitchen will be too small. You have to give every chef their own kitchen.

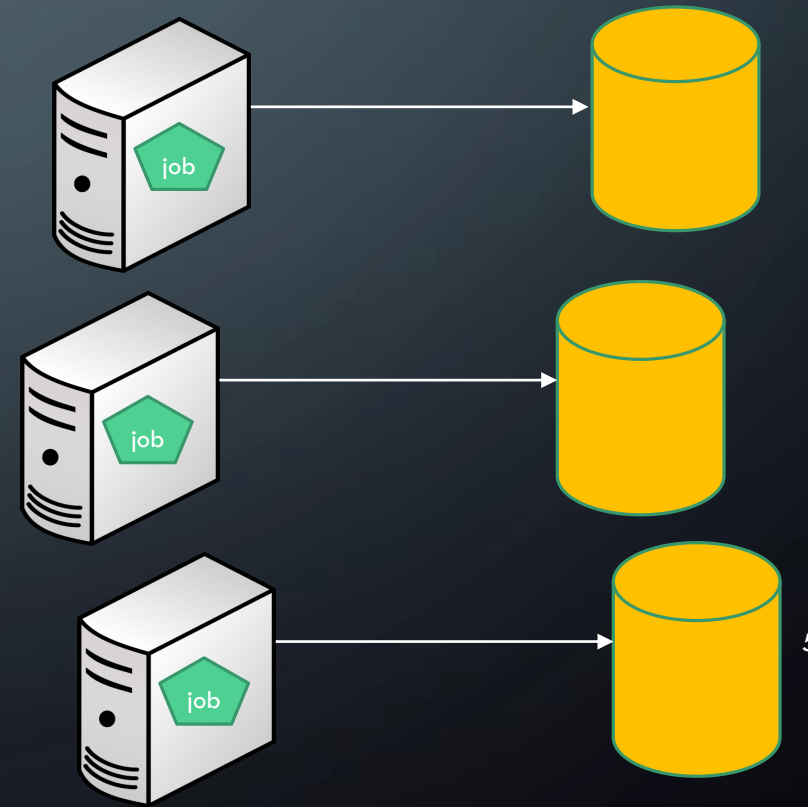


PARALLEL AND DISTRIBUTED PROCESSING

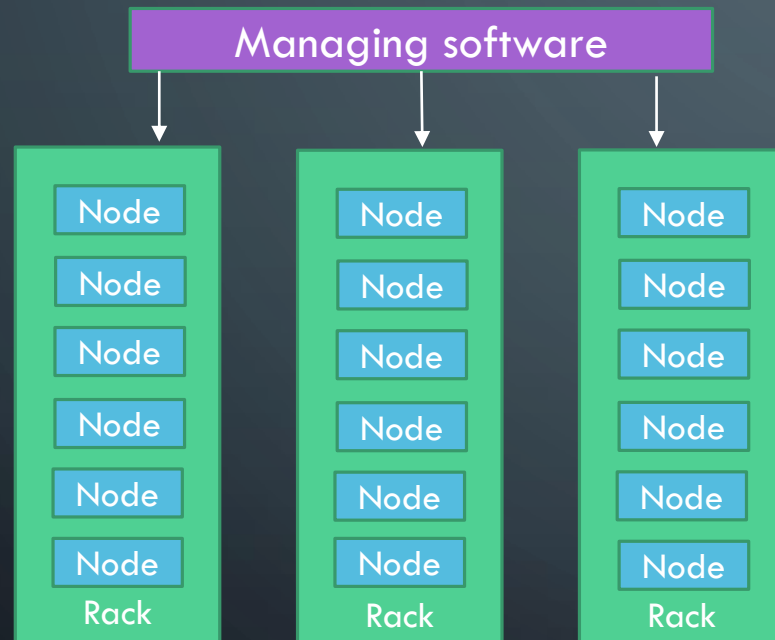
- Parallel



- Parallel and distributed



HOW TO MANAGE COMPUTING RESOURCES?



"Commodity hardware"



HADOOP

Storage and parallel processing on a massive scale



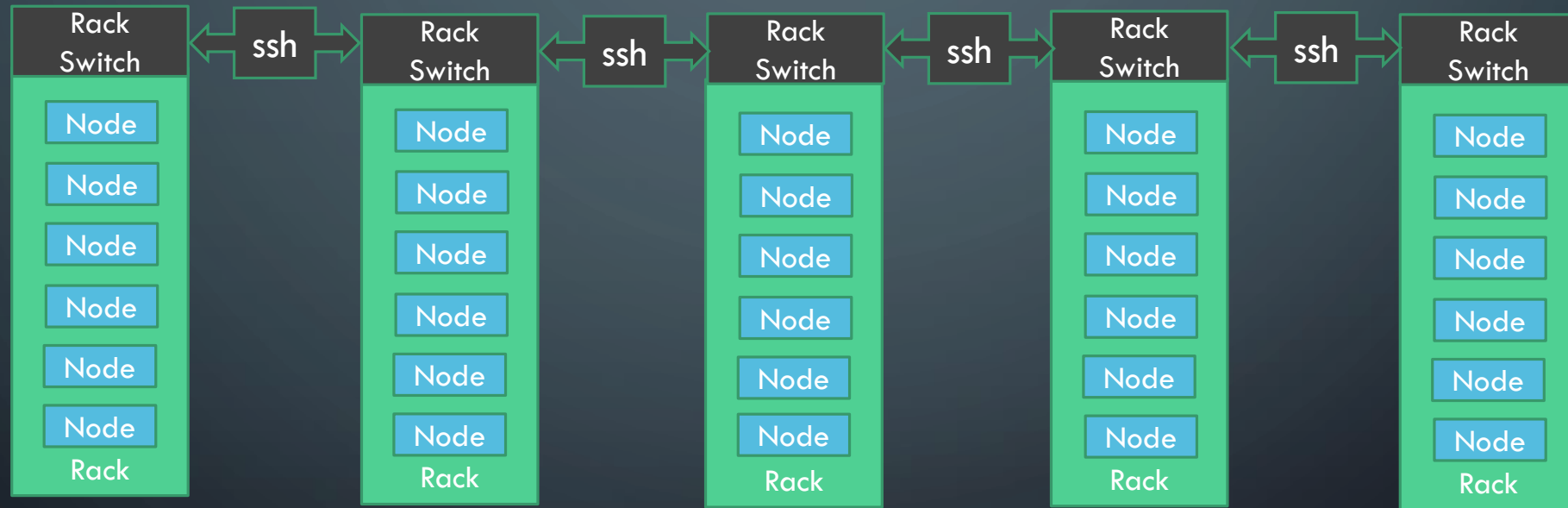
WHAT IS HADOOP?

An open source **software platform** for **distributed storage** and **distributed processing** of **very large data sets** on **computer clusters** built from commodity hardware."

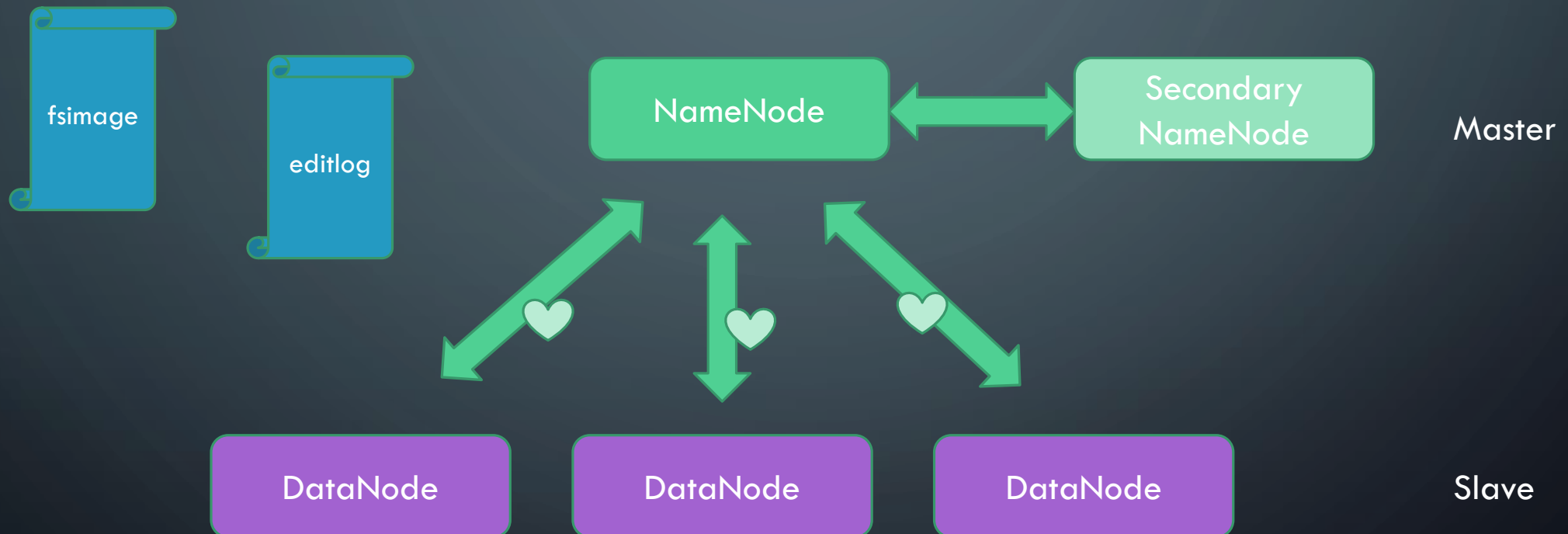
Core Hadoop:

MapReduce
Yarn
HDFS

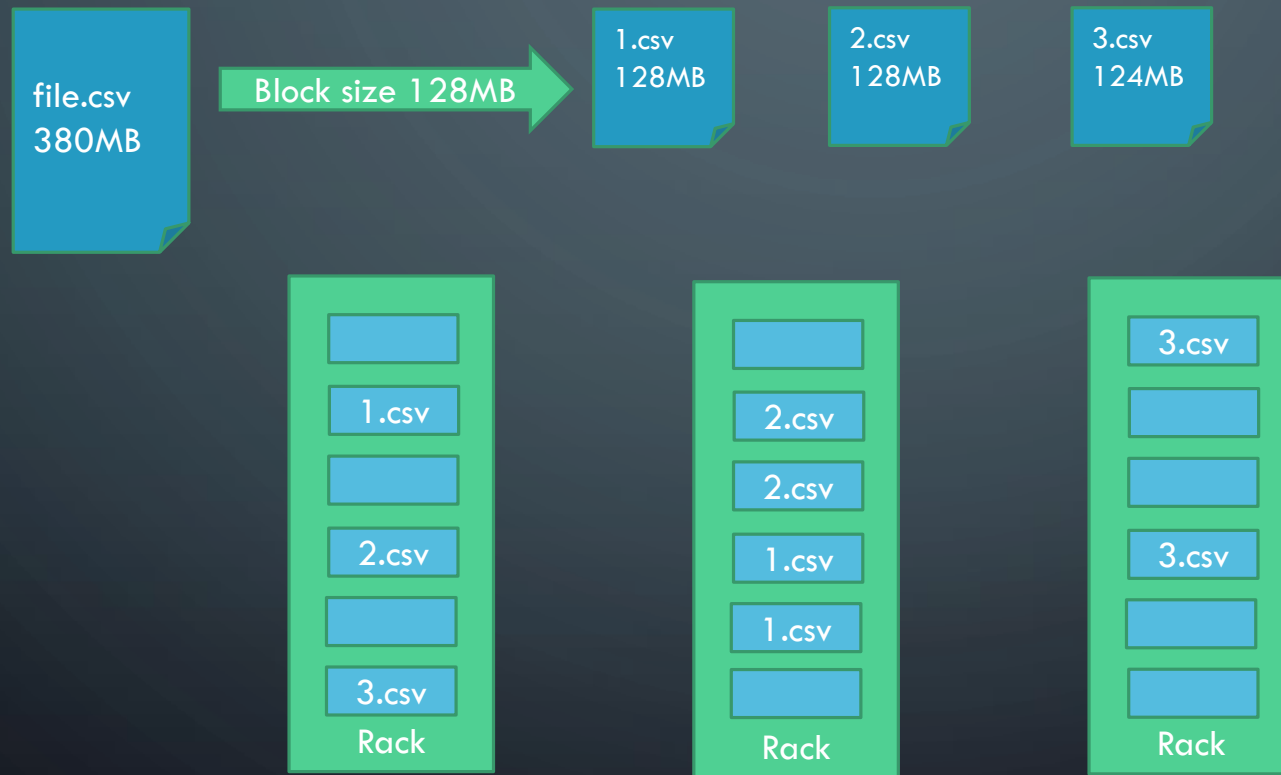
HADOOP CLUSTER



HADOOP DISTRIBUTED FILE SYSTEM - HDFS



HDFS STORAGE PROCESS



Default # replicas: 3

HDFS – LOGICAL VIEW

- Massively distributed file system provides flexible storage

/usr/container/datastore/

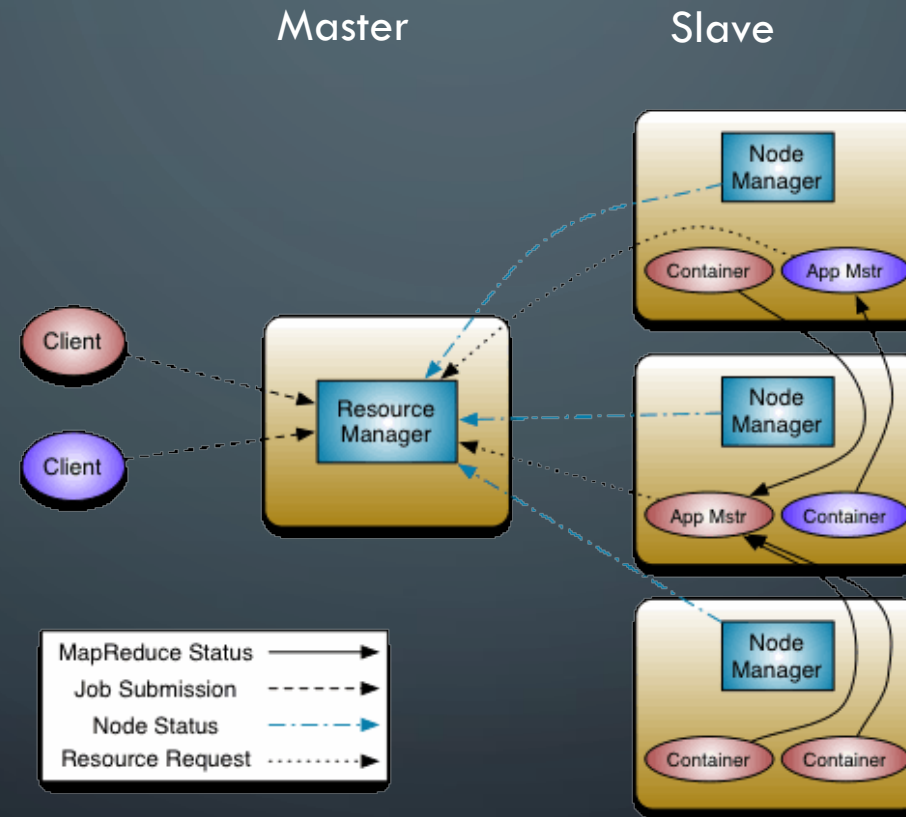
part_00001

part_00002

part_00003

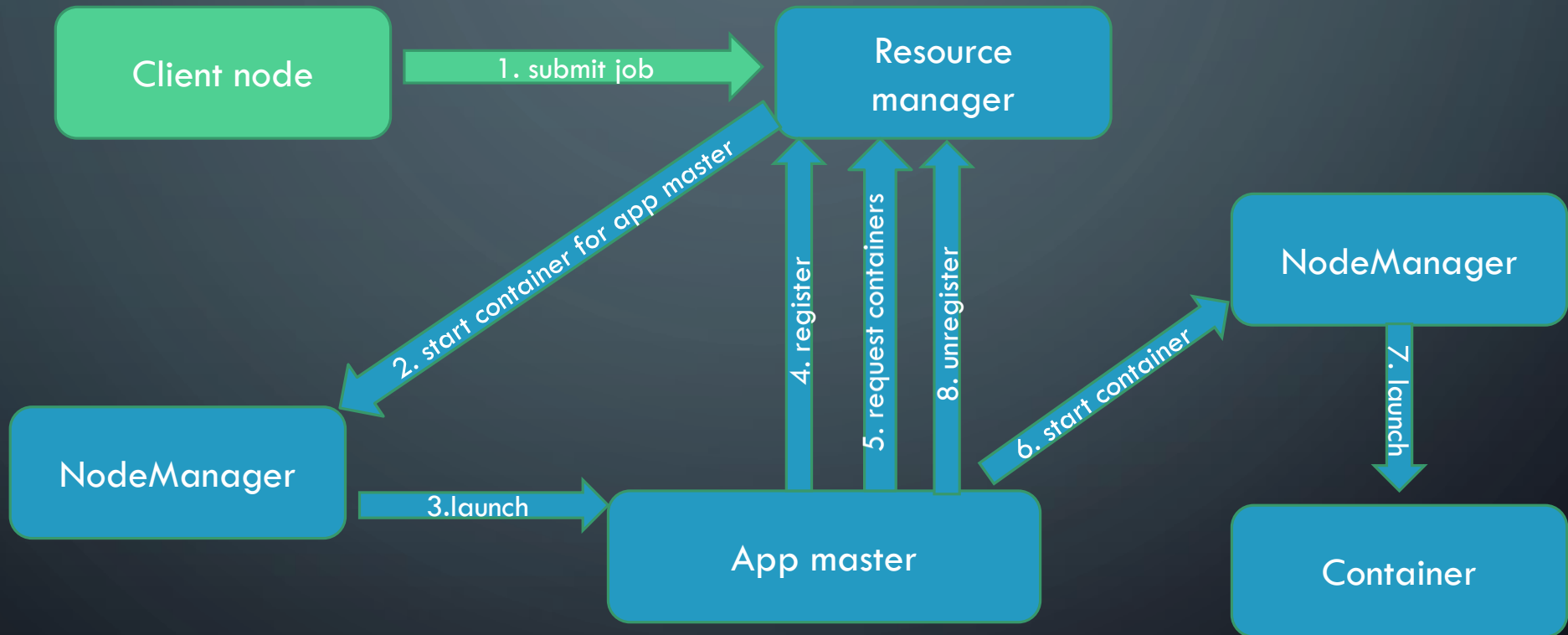
part_00004

HADOOP YARN



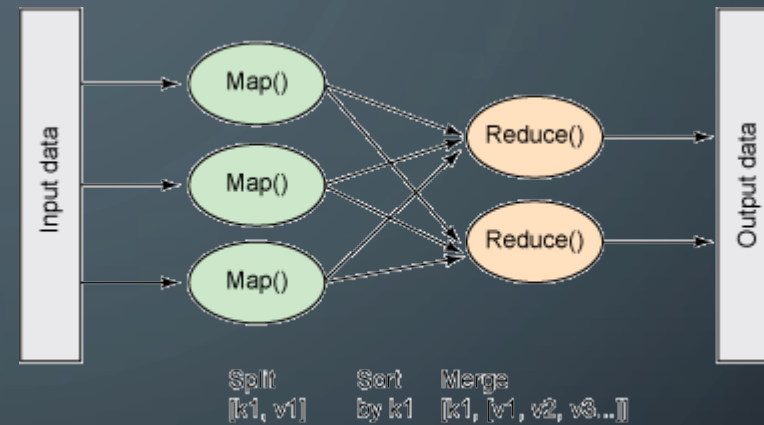
Source: apache.org

HADOOP YARN

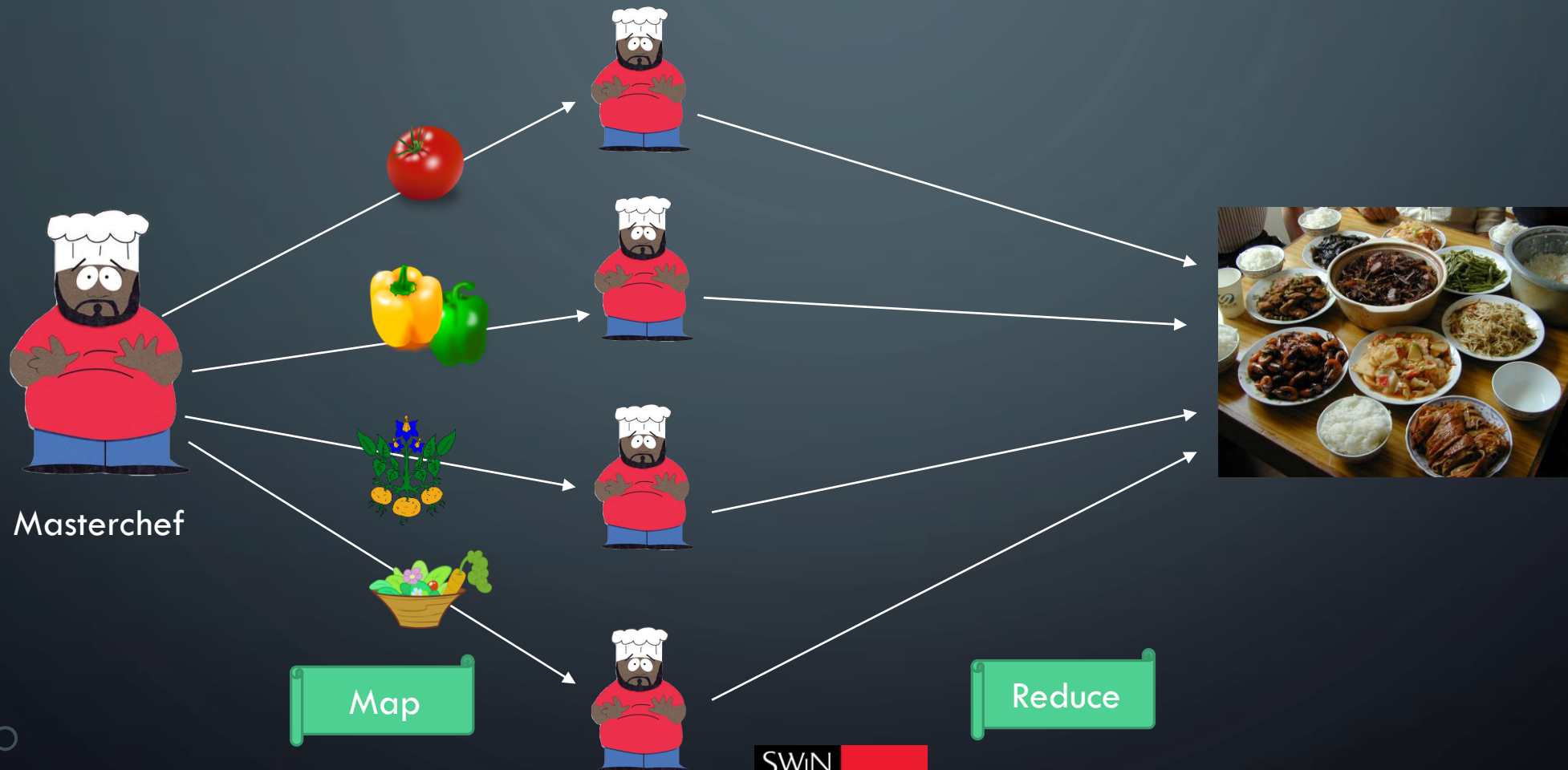


MAPREDUCE

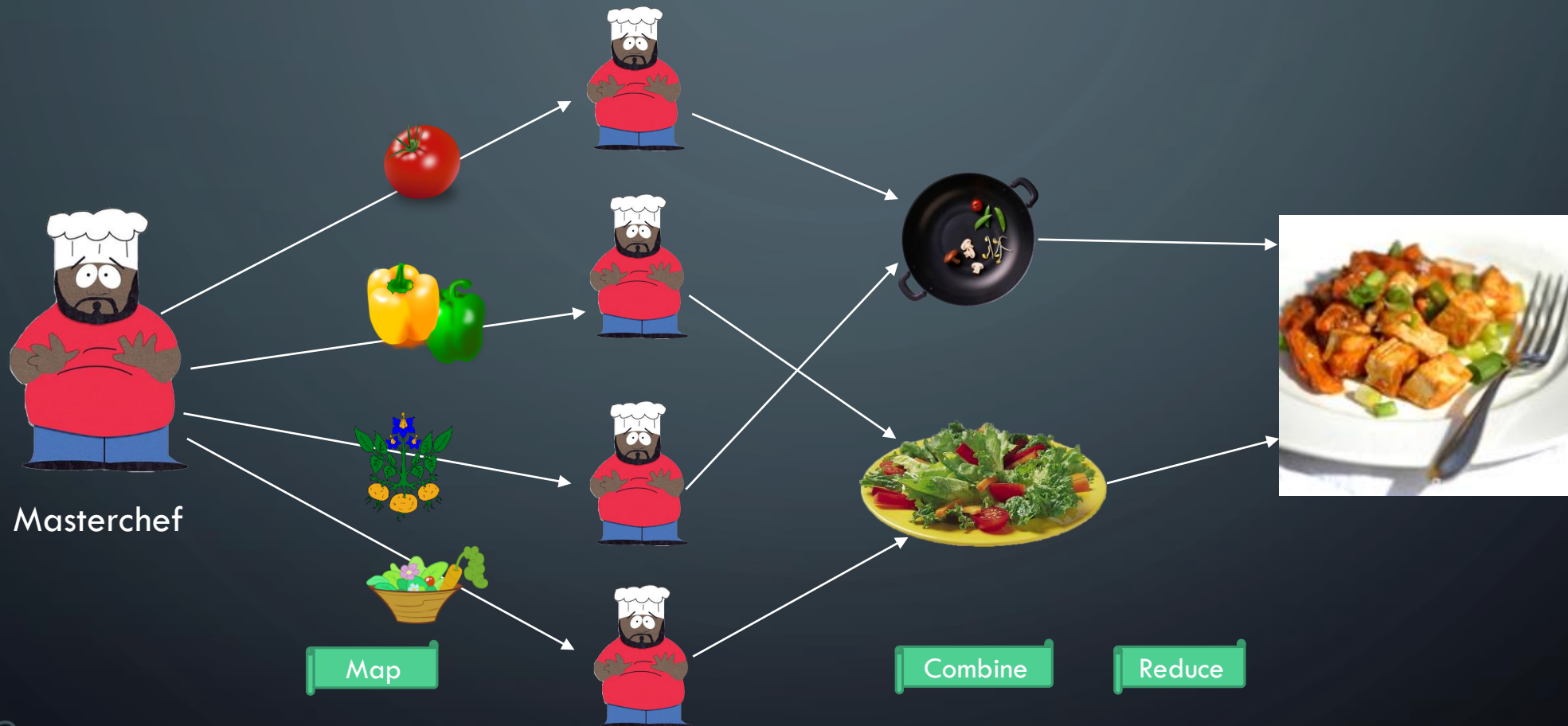
deserves its own section



EXAMPLE: PREPARING MEAL



EXAMPLE: PREPARING MEAL



HADOOP MAPREDUCE

SEQUENTIAL

```
for (i=0; i<x; i++)  
{  
    result[i] = read(list[i]);  
}
```



PARALLEL

```
result[0] = container[0].read(list[0]);  
result[1] = container[1].read(list[1]);  
result[2] = container[2].read(list[2]);  
...  
result[x] = container[x].read(list[x]);
```

Map



HADOOP MAPREDUCE

MAP

`result[0] = container[0].read(list[0]);`

`result[1] = container[1].read(list[1]);`

`result[2] = container[2].read(list[2]);`

...

`result[x] = container[x].read(list[x]);`

REDUCE

`result =`

`result[0] + result[1] +`

`result[2] +`

...

`result[x] ;`

ANOTHER MAPREDUCE EXAMPLE

Product	Customer	Location	Total
101043	8803085	Melbourne	12000
101044	8803086	Adelaide	6000
101045	8803087	Mumbai	76000

- We have a dataset that shows our sales in different locations.
- We want to sum the totals per location.
- Can we do this with MapReduce?

MAPREDUCE INPUT

Reside on different HDFS datanodes

101043	8803085	Melbourne	12000
101044	8803086	Adelaide	6000
101045	8803087	Mumbai	76000

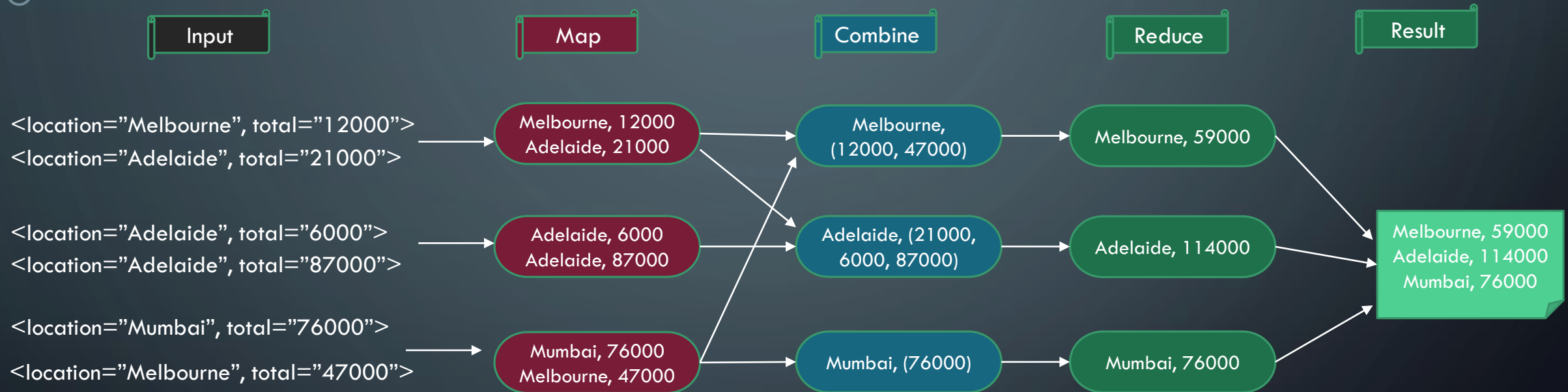
Transform to key-value pairs

<location="Melbourne", total="12000">

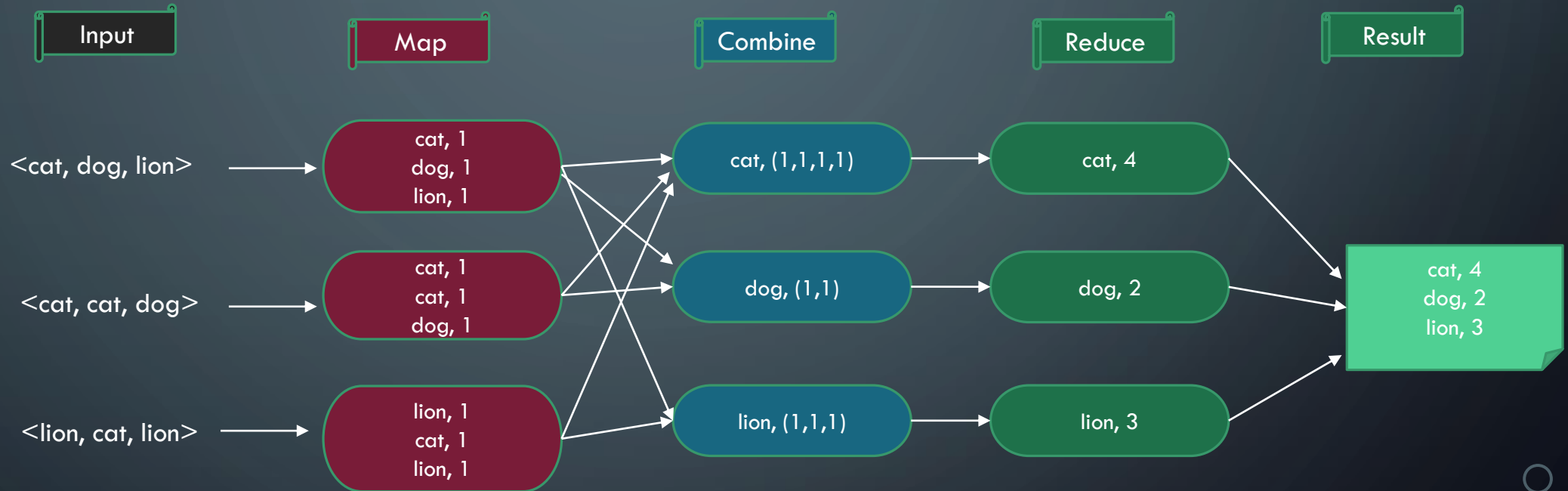
<location="Adelaide", total="6000">

<location="Mumbai", total="76000">

MAPREDUCE PROCESS



FAMOUS MAPREDUCE WORDCOUNT EXAMPLE



DRIVER – IN JAVA CODE

Starts the program

```
public static void main(String[] args) throws Exception {  
  
    Configuration conf= new Configuration();  
  
    Job job = new Job(conf,"The famous Word Count example");  
    job.setJarByClass(WordCount.class);  
    job.setMapperClass(Map.class);  
    job.setReducerClass(Reduce.class);  
    job.setOutputKeyClass(Text.class);  
    job.setOutputValueClass(IntWritable.class);  
    job.setInputFormatClass(TextInputFormat.class);  
    job.setOutputFormatClass(TextOutputFormat.class);  
  
    //Adding locations for the source and result files  
    FileInputFormat.addInputPath(job, new Path(args[0]));  
    FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
    //exiting the job only if the flag value becomes false  
    System.exit(job.waitForCompletion(true) ? 0 : 1);  
}
```

Tell the job what mapper and reducer to use

Define key and value types for input and output

Tell the job where to find the source file and where to put the result

Start the job

MAPPER – IN JAVA CODE

Input key/value and
output key/value

cat, 1
dog, 1
lion, 1

```
public static class Map extends Mapper<LongWritable,Text,Text,IntWritable> {  
    public void map(LongWritable key, Text value,Context context) throws IOException,InterruptedException{  
        String line = value.toString();  
        Text outputKey = new Text();  
        StringTokenizer tokenizer = new StringTokenizer(line);  
        while (tokenizer.hasMoreTokens()) {  
            outputKey.set(tokenizer.nextToken());  
            context.write(outputKey, new IntWritable(1));  
        }  
    }  
}
```

The first two values are
the actual input key and
value

The context variable
(argument) is used to
pass values to the
reducer

The actual work happens here.
StringTokenizer is used extensively in MR.

File.txt

line	text
0	The quick brown
277	at no extra cost
487	was well placed to

REDUCER – IN JAVA CODE

The output of the mapper is the input of the reducer

```
public static class Reduce extends Reducer<Text,IntWritable,Text,IntWritable> {  
    public void reduce(Text key, Iterable<IntWritable> values,Context context) throws IOException,InterruptedException {  
        int sum=0;  
        for(IntWritable x: values)  
        {  
            sum+=x.get();  
        }  
        context.write(key, new IntWritable(sum));  
    }  
}
```

Actual input values are passed to the method

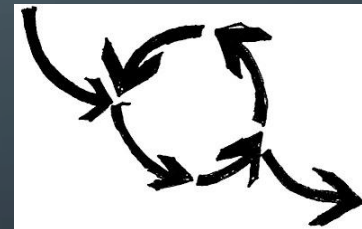
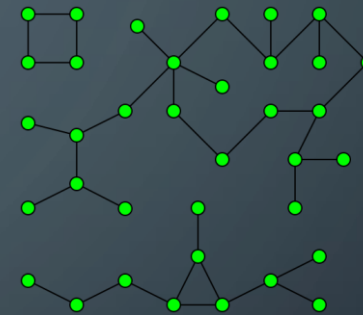
Actual work happens here.

Write result to context

cat, 4

LIMITATIONS OF MAPREDUCE

- No real-time processing
- Unsuitable for networked data
- Unsuitable for iterative processing
- Unsuitable for processing graphs



SUMMARY

- The potential bottlenecks for the speedy processing of big data are storage, processor and memory.
- The solutions are parallelisation and distribution.
- Hardware has moved into the racks of Cloud data centres
- Hadoop is the most prominent approach to managing such an architecture.
- HDFS is a distributed file system, and Yarn is a distributed processing system.
- MapReduce is a parallel and distributed programming framework.
- Mappers execute in parallel and reducers coordinate the result.